

EIGEN: Efficient and Generalizable Vector Database Tuning with Mini-DBs and LLM-Guided Optimization

Yiyan Li^{*}, Guanli Liu[†], Renata Borovica-Gajic[‡], Andreas Kipf[‡], Cuiping Li^{*}, Hong Chen^{*}

^{*} Renmin University of China, [†] The University of Melbourne, [‡] University of Technology Nuremberg
liyiyi, licuiping, chong@ruc.edu.cn; guanli.liu1, renata.borovica@unimelb.edu.au; andreas.kipf@utn.de

ABSTRACT

Vector databases support modern AI applications such as semantic search, recommendation systems, and retrieval-augmented generation. Their performance is highly sensitive to configuration choices. However, configurations that achieve high query throughput while satisfying recall requirements can vary across database engines, workloads, and hardware environments. Identifying such configurations requires repeated and costly evaluations on the full database.

We present μ -Tune, an efficient and generalizable framework for vector database tuning. To reduce the cost of configuration evaluation, μ -Tune constructs multiple small yet representative MiniDBs using manifold-preserving sampling, enabling efficient and parallel tuning. To reduce the number of evaluations, μ -Tune further introduces CMOOLL, a constraint-aware multi-objective optimizer based on LLMs. It uses knob semantics and feedback from previous evaluations to guide the search toward high-throughput configurations that satisfy the target recall constraint. Without task-specific pre-training, μ -Tune can adapt to different database engines, workloads, and hardware environments. Across multiple vector databases, μ -Tune reduces end-to-end tuning time by 98.87% on average while satisfying the recall constraint compared with the strongest baseline.

PVLDB Reference Format:

Yiyan Li^{*}, Guanli Liu[†], Renata Borovica-Gajic[‡], Andreas Kipf[‡], Cuiping Li^{*}, Hong Chen^{*}. EIGEN: Efficient and Generalizable Vector Database Tuning with Mini-DBs and LLM-Guided Optimization. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Vector databases underpin modern AI applications such as semantic search [12], recommendation systems [28], and retrieval-augmented generation (RAG) [15]. Achieving high query throughput while maintaining retrieval quality requires careful configuration across multiple layers of the system [33]. These configuration choices broadly span three categories: system knobs, index configurations, and search parameters. *System knobs* govern resource

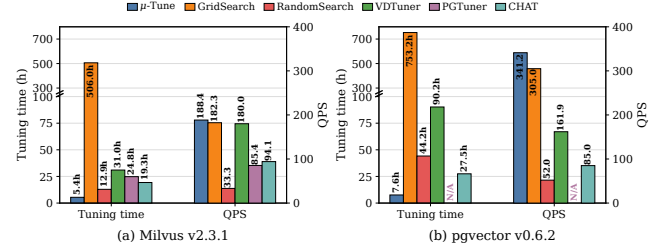


Figure 1: Comparison of end-to-end tuning time and QPS on Tiny5M under recall constraint of 0.9. N/A means a method failed to transfer to the corresponding database engine.

allocation and execution behavior, such as memory usage and query parallelism [30]. *Index configurations* determine how the index is constructed, such as graph connectivity, partition granularity, and compression settings [12, 19]. *Search parameters* control query-time exploration, such as the search width in HNSW, directly affecting the trade-off between QPS and recall [14, 19]. Given a database engine, workload, and hardware environment, vector database tuning seeks an end-to-end configuration across these layers that maximizes QPS while satisfying a target recall constraint [33].

Manually solving this tuning problem is challenging even for experienced database administrators (DBAs). The configuration space is mixed, hierarchical, and highly interdependent: selecting an index type activates a different set of parameters, while the effect of each parameter depends on the remaining configuration, dataset, workload, and hardware environment [30, 33]. These complex interactions make it difficult for DBAs to reason about parameter effects or identify promising regions of the configuration space. Moreover, tuning experience gained from one deployment is difficult to transfer reliably to another [8, 33]. As a result, manual tuning typically relies on expert knowledge and iterative trial and error, with no guarantee of finding a high-performance configuration that satisfies the target recall constraint.

To reduce reliance on manual trial and error, automated tuning methods have been proposed for vector databases. Search-based methods explore a predefined set of candidate configurations and select the best observed one [32, 36]. Bayesian optimization (BO)-based methods build a surrogate model from previous evaluations and use it to select promising configurations for subsequent trials [33, 35]. Reinforcement learning (RL)-based methods instead learn a configuration policy through repeated interaction with the database environment [3, 8].

Despite substantial progress in automated tuning, existing approaches still face two key limitations in practical vector database

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: Generalizability of vector database tuners.

Method	Tuning scope	Index families	Database engines	Offline dependence	Adaptation overhead
CHAT [5]	I+Q	HNSW	Multiple	Yes	Medium
PGTuner [8]	I+Q	Graph	Limited	Yes	Very high
VDTuner [33]	S+I+Q	Multiple	Milvus	No	High
EIGEN	S+I+Q	Multiple	Multiple	No	Low

S: system knobs; I: index configurations; Q: search parameters.

deployments: **low tuning efficiency** and **limited generalizability**. Figure 1 illustrates both issues on the Tiny5M dataset. On Milvus [31], existing methods require substantially longer tuning time than μ -Tune to obtain competitive configurations (e.g., 506.0 hours for GridSearch vs. 5.4 hours for EIGEN). When moving to pgvector [23], the gap becomes even larger: some methods require significantly more tuning time while achieving lower QPS (e.g., VDTuner requires 12 $\times$ the tuning time of EIGEN yet achieves 52.5% lower QPS), whereas PGTuner [8] cannot support the engine.

The efficiency issue stems from two factors: the cost of each configuration evaluation and the number of evaluations required. Existing methods evaluate candidate configurations directly on the full database, repeatedly rebuilding indexes and replaying the target workload, which makes each evaluation expensive [5, 32, 33]. At the same time, many evaluations are typically required to find a high-quality configuration: search-based methods explore numerous candidates. For example, GridSearch evaluates 4068 configurations for each dataset on Milvus. BO- and RL-based methods likewise rely on repeated sampling and performance feedback to update their optimization models, often requiring hundreds of iterations before identifying a high-quality configuration. Together, expensive evaluations and a large number of iterations lead to prohibitively long end-to-end tuning time.

The generalizability limitations of existing tuners span several dimensions, as summarized in Table 1. First, their tuning scope and index-family coverage are often restricted: CHAT [5] and PGTuner [8] focus on index and search parameters and are specialized to HNSW or graph-based indexes, whereas VDTuner [33] provides broader coverage. Second, only CHAT [5] supports multiple backends, while PGTuner [8] has limited engine support and VDTuner [33] is designed for Milvus. Finally, existing methods may depend on offline components or incur substantial adaptation overhead when applied to a new deployment. Together, these limitations make it difficult to reuse the same tuning procedure effectively across heterogeneous environments.

To address these limitations, we propose μ -Tune, an efficient and generalizable framework for vector database tuning. For tuning efficiency, μ -Tune constructs multiple small yet representative MiniDBs using manifold-preserving sampling and tunes them in parallel, substantially reducing the cost of index construction and workload execution. It further introduces CMOOLL, a constraint-aware multi-objective optimizer that uses LLMs to leverage knob semantics and historical evaluation feedback, reducing the number of evaluations needed to identify promising configurations that satisfy the recall constraint. For generalizability, μ -Tune separates engine-specific configuration descriptions and execution interfaces

from its core tuning procedure, allowing the same optimizer to be reused across different database engines and configuration spaces.

In summary, we make the following contributions:

- We present μ -Tune, an efficient and generalizable framework for end-to-end vector database tuning. It jointly tunes system knobs, index configurations, and search parameters to maximize query throughput under user-specified recall constraints.
- We develop a MiniDB-based tuning strategy that uses manifold-preserving sampling to construct multiple small yet representative databases. By performing evaluations on MiniDBs in parallel and transferring only promising candidates to the full database, μ -Tune substantially reduces the evaluation cost.
- We develop CMOOLL, a constraint-aware multi-objective optimizer that uses LLMs to reason over knob semantics and historical evaluation feedback, reducing the number of evaluations required to identify promising recall-feasible configurations. We further provide theoretical guarantees for MiniDB-based candidate selection and the asymptotic consistency of CMOOLL.
- We design a reusable tuning architecture that decouples engine-specific configuration descriptions and execution interfaces from the core tuning procedure, enabling adaptation across database engines, workloads, and hardware environments without task-specific offline training or model fine-tuning. Extensive experiments across multiple database engines demonstrate that μ -Tune achieves superior tuning quality while substantially reducing end-to-end tuning time.

2 BACKGROUND AND RELATED WORK

Vector Databases. Vector databases are designed to support efficient similarity search over high-dimensional vectors using approximate nearest-neighbor (ANN) indexes. A variety of vector database systems have been developed with different architectures and design choices. Rather than exhaustively covering all available systems, we focus on three representative and commonly used systems: Milvus [31], Qdrant [?], and pgvector [23]. Milvus and Qdrant are purpose-built vector database systems with specialized indexing and data-management components, whereas pgvector integrates vector search into the general-purpose PostgreSQL DBMS. These architectural differences result in substantially different configuration interfaces and execution behaviors.

Tuning Space. Despite their architectural differences, tunable configurations in vector databases can be broadly organized into three layers. *System knobs* control resource allocation and execution behavior, such as memory management, parallelism, and background maintenance. *Index configurations* determine the index family and its construction parameters; for example, HNSW exposes M and $efConstruction$, whereas IVF-based indexes expose parameters such as $nlist$ and compression settings. *Search parameters* control query-time exploration, such as ef for HNSW and $nprobe$ for IVF. The concrete tuning space is engine-dependent: Milvus supports multiple HNSW-, IVF-, and SCANN-based indexes, Qdrant combines HNSW and quantization with segment and storage configurations, while pgvector exposes vector-index parameters together

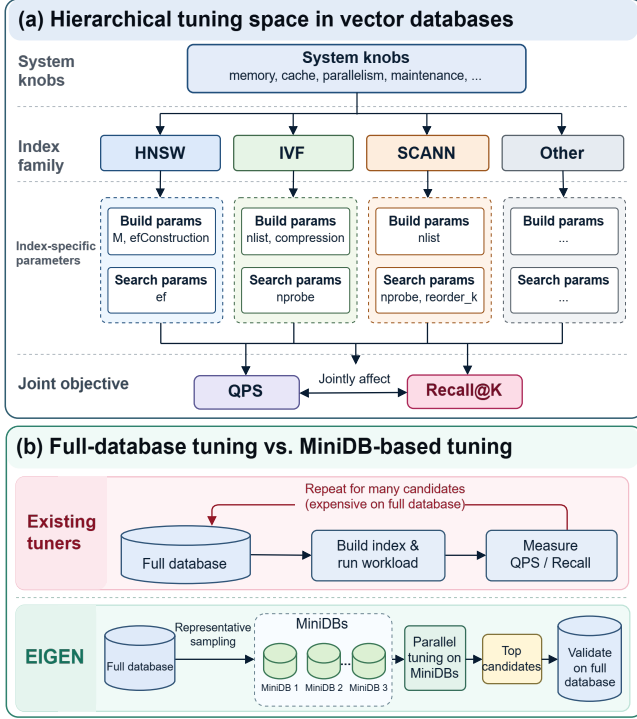


Figure 2: Comparison of tuning effectiveness and efficiency.

with PostgreSQL system knobs. Furthermore, the space is hierarchical: selecting an index family determines which construction and search parameters become active.

Vector Database Tuning. Vector database tuning shares the same general goal as conventional DBMS tuning: selecting configurations that improve workload performance based on runtime feedback. Traditional database tuning primarily adjusts system knobs to optimize throughput or latency while query semantics and result correctness remain unchanged. Vector database tuning, however, differs in two important aspects. First, ANN search deliberately trades retrieval quality for efficiency [12, 19, 31], so index and search configurations can simultaneously affect both QPS and Recall@K. Given a configuration space C and target recall ρ , we seek

$$c^* = \arg \max_{c \in C} \text{QPS}(c) \quad \text{s.t.} \quad \text{Recall@K}(c) \geq \rho. \quad (1)$$

Second, its tuning space spans system, index, and search layers and is hierarchical, as selecting an index family activates different parameters.

These differences also make existing learned tuners difficult to reuse across deployments. Such methods typically learn configuration-performance relationships tied to particular configuration spaces, workloads, and environments. Changes in database engines can alter parameter sets and semantics, while workload or hardware changes can alter their performance effects. Consequently, transfer may require additional measurements, model adaptation, or retraining.

Search-based Tuning. Search-based methods directly evaluate candidate configurations and retain the best observed ones. GridSearch [32] discretizes parameter domains and enumerates their Cartesian product. It provides a strong reference for tuning quality when the grid is sufficiently dense, but the number of evaluations grows rapidly with the size and dimensionality of the configuration space. RandomSearch reduces this cost by evaluating only a sampled subset of configurations, but can miss high-quality regions in large heterogeneous spaces. For vector databases, CHAT [5] exploits HNSW-specific structural properties and resource-feasibility pruning to reduce unnecessary evaluations. Its specialization enables effective HNSW tuning across multiple backends, but does not directly extend to other ANN families or system-level knobs.

BO-based Tuning. Bayesian optimization (BO) models configuration performance with a surrogate and uses an acquisition function to select promising configurations for subsequent evaluation [20, 30, 35]. Compared with exhaustive search, BO can substantially reduce the number of physical evaluations by exploiting observations collected during tuning. VDTuner [33] applies multi-objective BO to vector database tuning and jointly explores system, index, and search parameters while considering the trade-off between search efficiency and recall. However, its tuning space and implementation are designed around Milvus, making direct reuse on engines with different parameter spaces and execution interfaces difficult.

RL- and Learned-based Tuning. Reinforcement learning (RL)-based methods learn configuration policies from performance feedback [16, 29, 34]. These methods can exploit experience collected during tuning to navigate large configuration spaces more efficiently. For vector indexes, PGTuner [8] combines a pre-trained performance predictor with an RL-based configuration recommender. The predictor reduces physical evaluations by estimating configuration performance, while the recommender adapts configurations to a target dataset. However, its tuning scope is restricted to proximity-graph indexes such as HNSW and NSG, and applying the learned models to new settings introduces training-data collection and adaptation costs. More importantly, learned components may rely on engine-specific features or performance signals that are unavailable after switching database engines.

Summary. Figure ?? highlights two characteristics that motivate our design. First, vector database tuning requires navigating a hierarchical configuration space spanning system, index, and search layers. Second, reliable performance feedback is commonly obtained through physical index construction and workload execution on the full database, making each evaluation costly. μ -Tune addresses these two aspects by combining a reusable, semantics-guided optimizer with representative MiniDBs that move most configuration evaluations away from the full database.

3 OVERVIEW

Given a vector database, a query workload, and a target recall constraint, EIGEN jointly tunes system knobs, index configurations, and search parameters to maximize QPS while satisfying the recall requirement. As shown in Figure 3, the workflow consists of three phases: *data abstraction*, *parallel optimization*, and *global selection*.

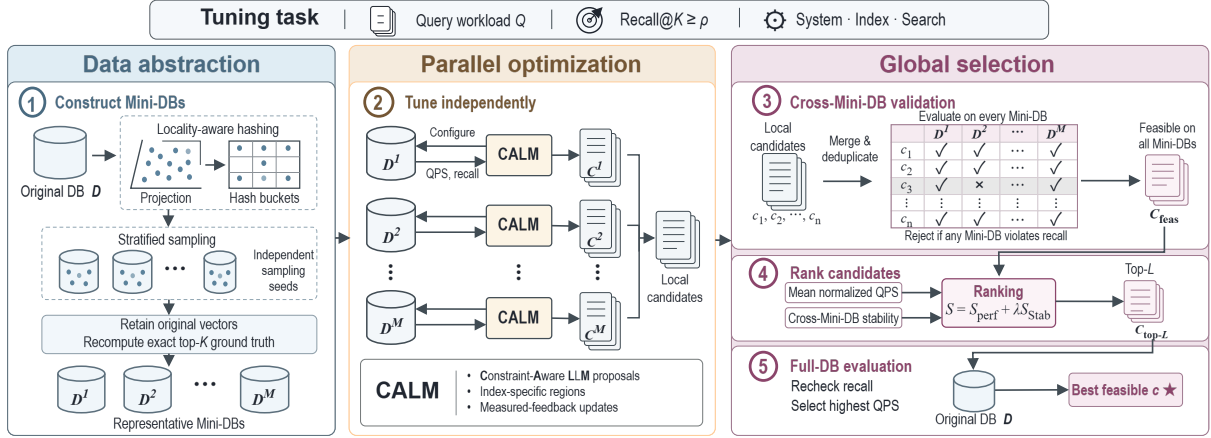


Figure 3: End-to-end workflow of EIGEN: data abstraction with representative MiniDBs, parallel optimization with CALM, and global selection with final full-scale evaluation.

3.1 Data Abstraction

The first phase constructs multiple representative MiniDBs from the original database (Stage ①). EIGEN uses locality-aware stratified sampling to generate smaller databases that preserve the local structure of the original data. Because sampling changes the available nearest neighbors, exact Top- K ground truth is recomputed for each MiniDB, making each one a self-contained environment for configuration evaluation.

MiniDBs are introduced to reduce the cost of each configuration evaluation. Direct evaluation on the full database can be expensive because each trial may require index construction and workload execution over the complete dataset. By shifting most evaluations to much smaller databases, MiniDBs substantially reduce this cost, while multiple independently sampled MiniDBs reduce reliance on any single sample. Section 4 provides further details.

3.2 Parallel Optimization

The second phase tunes the MiniDBs independently and in parallel under the same objective and recall constraint (Stage ②). Each MiniDB is optimized by CALM, our constraint-aware LLM-guided optimizer. CALM uses configuration semantics, validity rules, historical evaluations, and measured feedback to identify promising configurations. Each tuning instance produces a local candidate set.

CALM is designed to reduce the number of physical evaluations. The vector database tuning space is large, hierarchical, and engine-specific, making exhaustive search inefficient and purely data-driven optimization costly. By combining semantic guidance with measured database feedback, CALM focuses the evaluation budget on promising regions while retaining exploration. Section 5 provides further details.

3.3 Global Selection

The final phase combines the candidate sets produced by all MiniDBs. EIGEN first merges and deduplicates the candidates and evaluates each one across all MiniDBs (Stage ③). Configurations violating the recall constraint are removed, and the remaining candidates are ranked according to their average normalized performance and cross-MiniDB stability (Stage ④). Only the top- L candidates

are transferred to the original database for full-scale evaluation, where the feasible configuration with the highest QPS is returned (Stage ⑤).

4 MINIDB-BASED TUNING FRAMEWORK

This section presents our MiniDB-based tuning framework, which reduces full-database evaluation costs by conducting most configuration searches on multiple smaller databases. Algorithm 1 summarizes the workflow. We first construct MiniDBs through locality-aware stratified sampling, tune each MiniDB independently, and evaluate the resulting candidates across MiniDBs. We then transfer a small set of promising configurations to the original database for final validation. Section 4.3 analyzes the geometric coverage of the construction and the reliability of cross-MiniDB evaluation.

4.1 Locality-Aware MiniDB Construction

Given a database $\mathcal{D}$ containing N vectors, we construct M MiniDBs, each containing $n \ll N$ original vectors. Uniform sampling can leave regions containing few vectors unrepresented. Our construction uses geometric partitions and proportional sampling to retain coverage across these regions while approximately preserving their relative populations.

Metric-compatible locality-sensitive partitioning. MiniDB construction is parameterized by a locality-sensitive partition compatible with the workload similarity metric. Below, we instantiate the partition using random-hyperplane locality-sensitive hashing (LSH) [4] for angular/cosine similarity. Each nonzero vector is mapped to the signs of its projections onto random Gaussian directions. The resulting binary signature defines a hash bucket: vectors with smaller angles are more likely to share a signature. Euclidean workloads require a partitioner compatible with Euclidean distance; the sampling, tuning, and aggregation procedures remain the same. Hashing is used only to select vector identifiers, while index construction, ground-truth computation, and query evaluation use the original vectors and workload metric.

Bucket coverage and proportional sampling. We generate one ordered sequence of hash functions up to a prescribed maximum

Algorithm 1 MiniDB-Based Tuning Framework

Require: Database $\mathcal{D}$, queries $\mathcal{Q}$, constraints $\mathcal{G}$; M MiniDBs of size n ; transfer budget L ; maximum signature length $b_{\max}$

Ensure: Best feasible configuration among validated candidates, or $\perp$

```
1:  $\mathcal{B} \leftarrow \text{HashBucketize}(\mathcal{D}, n, b_{\max})$ 
2: for  $m = 1, \dots, M$  do
3:    $\mathcal{D}^{(m)} \leftarrow \text{StratifiedSample}(\mathcal{D}, \mathcal{B}, n)$  using an independent seed
4:   Recompute exact top- $K$  ground truth on  $\mathcal{D}^{(m)}$ 
5:    $\mathcal{C}^{(m)} \leftarrow \text{Tune}(\mathcal{D}^{(m)}, \mathcal{Q}, \mathcal{G})$ 
6: end for
7:  $\mathcal{C} \leftarrow \bigcup_{m=1}^M \mathcal{C}^{(m)}$ 
8: Evaluate every  $c \in \mathcal{C}$  on all MiniDBs
9:  $\mathcal{C}_{\text{feas}} \leftarrow \text{ConstraintFilter}(\mathcal{C}, \mathcal{G})$ 
10: if  $\mathcal{C}_{\text{feas}} = \emptyset$  then
11:   return  $\perp$ 
12: end if
13: for each  $c \in \mathcal{C}_{\text{feas}}$  do
14:   Compute its mean normalized utility and standard deviation
15:    $S(c) \leftarrow S_{\text{perf}}(c) + \lambda S_{\text{stab}}(c)$ 
16: end for
17:  $\mathcal{C}_L \leftarrow \text{TopL}(\mathcal{C}_{\text{feas}}, S, L)$ 
18: return  $\text{FullScaleValidate}(\mathcal{C}_L, \mathcal{D}, \mathcal{Q}, \mathcal{G})$ 
```

length $b_{\max}$. Among its prefixes, we choose the largest signature length $b \leq b_{\max}$ for which the number of non-empty buckets does not exceed n . Appending a hash component refines the existing partition, so this selects the finest available partition compatible with minimum-one sampling. The empty prefix, corresponding to a single bucket, provides a feasible fallback. Following proportional stratified sampling [6], sample counts are allocated proportionally to bucket sizes, subject to three requirements: each non-empty bucket contributes at least one vector, no bucket contributes more vectors than it contains, and the total allocation equals n . Largest-remainder rounding is applied subject to these bounds. Within each bucket, vectors are sampled uniformly without replacement. The minimum-one rule ensures bucket coverage, although it can increase the relative representation of small buckets. The resulting bucket partition is shared across all MiniDBs, which use independent sampling seeds.

Ground-truth recomputation. For each MiniDB, we recompute the exact top- K neighbors of every query in $\mathcal{Q}$. The full-database ground truth cannot be reused because its nearest neighbors may be absent from the sample. Each MiniDB therefore has its own exact recall reference, and configurations are evaluated against the same query workload but the appropriate local ground truth.

4.2 Cross-MiniDB Aggregation and Full-Database Validation

Each MiniDB is tuned independently by the optimizer in Section 5, producing a candidate set $\mathcal{C}^{(m)}$. Each local proposal has an originating MiniDB before the candidate sets are pooled. We merge these

sets and remove duplicates to obtain $\mathcal{C} = \bigcup_{m=1}^M \mathcal{C}^{(m)}$. Every candidate is then evaluated on every MiniDB. For a proposal generated by source MiniDB s , the other $M - 1$ MiniDBs provide evaluation views that did not participate in its generation. These cross-checks expose configurations whose performance or feasibility is specific to their originating sampled view.

Constraint filtering. We retain a candidate only if it satisfies $\mathcal{G}$ on every MiniDB, yielding $\mathcal{C}_{\text{feas}}$. This conservative rule favors configurations with consistent MiniDB-level feasibility. It can also discard configurations that would be feasible on the original database; consequently, it is a screening rule rather than a test of full-database feasibility.

Performance and stability scoring. For each surviving candidate c , let $\widehat{J}_m(c) \in [0, 1]$ be its QPS on MiniDB m , min-max normalized over $\mathcal{C}_{\text{feas}}$ on that MiniDB. A lower-is-better objective would use the reversed normalization. Let $\widehat{J}_M(c)$ and $\sigma_J(c)$ denote the mean and standard deviation of these normalized utilities. We compute

$$\begin{aligned} S_{\text{perf}}(c) &= \text{Norm}(\widehat{J}_M(c)), \\ S_{\text{stab}}(c) &= 1 - \text{Norm}(\sigma_J(c)), \\ S(c) &= S_{\text{perf}}(c) + \lambda S_{\text{stab}}(c), \end{aligned} \tag{2}$$

where Norm denotes min-max normalization across the surviving candidates and $\lambda \geq 0$ controls the stability contribution. We use $\lambda = 1$ unless stated otherwise. When all values being normalized are equal, the normalized values are set to zero. The score favors high average performance and low sensitivity to the sampled database. The stability term is a robustness regularizer, not a calibrated confidence bound.

Full-database validation. We select up to L candidates with the highest scores and evaluate their recall and QPS on the original database. The framework returns the highest-QPS candidate that satisfies $\mathcal{G}$ under this full-database evaluation. If no candidate survives screening or full-database validation, it returns $\perp$, indicating that no feasible configuration was found within the evaluated candidate set and budget. Thus, MiniDBs prioritize expensive evaluations, while final acceptance depends on measurements at the target scale.

4.3 Theoretical Analysis

We analyze two aspects of the framework: the geometric coverage obtained by sampling every bucket, and the reliability of repeated MiniDB evaluations. The analysis distinguishes sampling variability from the systematic difference between MiniDB and full-database performance.

Locality and geometric coverage. Let d be a distance metric consistent with the workload's neighbor ordering: Euclidean distance for L_2 search, or normalized angular distance for cosine search. The coverage argument below applies to either choice with its corresponding partitioner. For the angular instantiation, let $d_{\text{ang}}(x, y) \in [0, 1]$ denote the angle between nonzero vectors divided by π . For a pre-specified signature length b and b independent random hyperplanes, the standard LSH collision identity is

$$\Pr[h(x) = h(y)] = (1 - d_{\text{ang}}(x, y))^b. \tag{3}$$

One hyperplane separates the vectors with probability $d_{\text{ang}}(x, y)$; independence gives the identity for b hyperplanes [4]. This explains the angular locality of the partition. Once the partition has been chosen to meet the sampling budget, the following coverage property holds deterministically.

LEMMA 1 (GEOMETRIC COVERAGE). *Let r be the largest diameter, measured using d , of any non-empty bucket in the chosen partition. If every bucket contributes at least one vector to $\mathcal{D}^{(m)}$, then*

$$\max_{x \in \mathcal{D}} \min_{y \in \mathcal{D}^{(m)}} d(x, y) \leq r. \quad (4)$$

PROOF. For any $x \in \mathcal{D}$, choose a sampled vector y from the same bucket. By the definition of r , $d(x, y) \leq r$. $\square$

Every original vector therefore has a representative within the resolution of its bucket under d . By the triangle inequality, the nearest-neighbor distance of a query in a MiniDB exceeds its full-database nearest-neighbor distance by at most r , under the same metric. The resolution r depends on the data and the chosen partition. This result characterizes geometric coverage; scale-dependent system performance is addressed separately below.

Independent validation of locally generated candidates. Consider a candidate $c \in C^{(s)}$ produced by tuning source MiniDB s using only that MiniDB and independent local randomness. We analyze this proposal before inspecting its outcomes on other MiniDBs. Conditioning on the original database, shared partition, workload, evaluation protocol, source MiniDB, and the local proposal or proposals being analyzed. Under independent sampling and execution randomness across MiniDBs, the other $M - 1$ MiniDBs remain independent evaluation views. All probabilities and expectations below use this conditioning.

For this analysis, $J_m(c) \in [0, 1]$ and $J(c) \in [0, 1]$ denote the measured MiniDB utility and the expected full-database utility on a common bounded scale fixed independently of the non-source views. This scale is used to analyze the validation measurements; Eq. 2 uses candidate-pool normalization for the final performance-stability ranking. For $M \geq 2$, define the mean of the non-source measurements already collected during cross-MiniDB evaluation:

$$\bar{J}_{-s}(c) = \frac{1}{M-1} \sum_{m \neq s} J_m(c). \quad (5)$$

Let β_n bound the transfer gap for each proposal being analyzed:

$$|\mathbb{E}[J_m(c)] - J(c)| \leq \beta_n, \quad m \neq s. \quad (6)$$

This term captures systematic MiniDB-to-full effects, including changes in index organization and resource behavior with database size.

THEOREM 1 (VALIDATION ON NON-SOURCE MINIDBS). *Under the preceding conditioning and assumptions, for a proposal $c \in C^{(s)}$ and $\delta \in (0, 1)$, with probability at least $1 - \delta$,*

$$|\bar{J}_{-s}(c) - J(c)| \leq \beta_n + \sqrt{\frac{\log(2/\delta)}{2(M-1)}}. \quad (7)$$

For two proposals c_1, c_2 generated by the same source MiniDB s , with full-database utility gap $\Delta = J(c_1) - J(c_2) > 2\beta_n$,

$$\Pr[\bar{J}_{-s}(c_1) \leq \bar{J}_{-s}(c_2)] \leq \exp\left(-\frac{(M-1)(\Delta - 2\beta_n)^2}{2}\right). \quad (8)$$

PROOF. Conditioning on the source MiniDB and its proposals fixes the configurations without conditioning on any non-source evaluation. Hoeffding's inequality [11] therefore applies to the $M - 1$ independent, bounded validation measurements. Combining its deviation bound with Eq. 6 gives Eq. 7. For the second claim, the differences $J_m(c_1) - J_m(c_2)$, $m \neq s$, are independent across MiniDBs, lie in $[-1, 1]$, and have conditional expectation at least $\Delta - 2\beta_n$. The one-sided Hoeffding bound yields Eq. 8. $\square$

The source MiniDB thus proposes configurations, while the other MiniDBs provide independent evidence of their behavior across sampled views. Increasing their number reduces sampling-induced uncertainty and, when the full-database utility gap exceeds $2\beta_n$, lowers the probability of misranking proposals from the same source. The transfer term depends on the MiniDB size and workload. The Fidelity of Mini Databases study (Section 6.5.1) assesses practical transfer across workloads through configuration rankings, recall errors, false-safe rates, and transfer regret. The Number and Size of Mini Databases study (Section 6.6.1) examines how n and M affect final configuration quality and variability. These experiments complement the error decomposition by evaluating whether the chosen sampling scale provides useful full-database candidates in practice.

The same source-conditioned reasoning applies to constraint filtering. Let $p(c)$ be the conditional probability that a proposal from source s violates a constraint on a new, independently sampled MiniDB. Independence of the $M - 1$ non-source checks gives

$$\Pr[\text{pass all } M - 1 \text{ non-source checks}] = (1 - p(c))^{M-1}. \quad (9)$$

Each check passes with probability $1 - p(c)$, so the probability of passing all checks is their product. A proposal whose feasibility depends on favorable sampled views is therefore less likely to survive repeated cross-checks.

Together, these results establish two roles of independent non-source MiniDBs: averaging reduces sampling-induced uncertainty in candidate evaluation, while repeated constraint checks suppress proposals whose feasibility is specific to favorable sampled views. The guarantees characterize independent validation evidence used in the screening stage, rather than full-scale performance equivalence. Systematic MiniDB-to-full transfer effects are captured separately by β_n , and final feasibility and QPS are determined by full-database validation.

5 CONSTRAINT-AWARE LLM-GUIDED OPTIMIZER

This section presents CALM, the constraint-aware LLM-guided optimizer used to tune each MiniDB. Inspired by region-based LLM optimization [17, 26], CALM combines index-specific search regions, constraint-aware scoring, and measured feedback to maximize QPS subject to a recall threshold. Section 5.1 describes the tuning procedure, and Section 5.2 analyzes its constrained optimality and Pareto-front consistency under sustained exploration.

5.1 Tuning Algorithm

Figure 4 summarizes the tuning workflow. Given a MiniDB, its query workload, an index catalog, a recall threshold ρ , and a tuning budget, CALM searches for high-QPS configurations whose

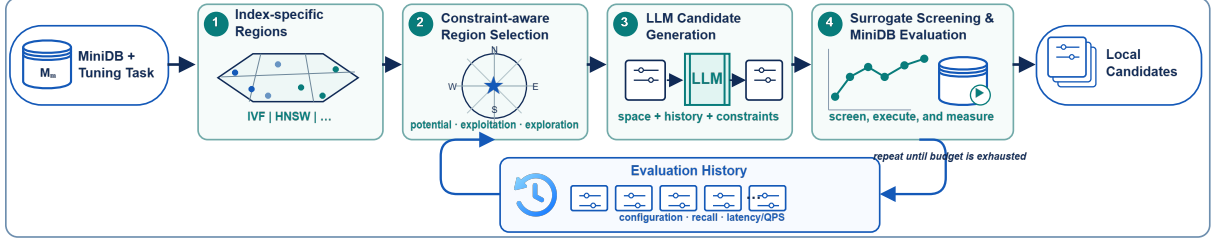


Figure 4: CALM tunes each MiniDB through index-specific region construction, constraint-aware region selection, LLM-based candidate generation, and measured feedback.

measured recall is at least ρ . It returns a feasible non-dominated archive for cross-MiniDB aggregation.

Search regions and initialization. CALM creates one region R_i for each supported index type $i \in \mathcal{I}$. Each region contains the applicable system knobs, index-construction parameters, and search parameters, together with their conditional validity rules. The admissible configuration space is the disjoint union

$$\mathcal{X} = \biguplus_{i \in \mathcal{I}} R_i. \quad (10)$$

This decomposition prevents invalid cross-index parameter combinations and gives the LLM a coherent local search space. CALM initializes its evaluation history and feasible archive using seed configurations. Within the archive, dominance is computed over measured QPS and recall, both to be maximized, after applying the recall constraint. These two coordinates support search guidance and candidate diversity; the tuning objective remains maximum QPS subject to recall feasibility.

Constraint-aware region selection. At each round, CALM scores the index-specific regions before allocating the next evaluation batch. For sufficiently explored regions, the score combines their contribution to the current feasible QPS–recall hypervolume with an exploration bonus. For new or under-explored regions, where reliable frontier estimates are unavailable, CALM uses estimated QPS improvement weighted by predicted recall-feasibility probability, following feasibility-weighted acquisition in constrained BO [9]. These scores guide evaluation allocation toward promising regions while accounting for the recall requirement. CALM then selects regions stochastically according to their scores, giving higher-scoring regions more opportunities for exploration.

Candidate generation. For each selected region, CALM constructs a prompt containing the VectorDB tuning task, the selected index schema, valid parameter ranges, conditional constraints, the recall threshold, the QPS objective, and representative measured evaluations. The LLM proposes configurations within that region. A deterministic validator removes malformed, duplicated, and schema-invalid proposals before adding the remaining configurations to the candidate pool.

Surrogate selection and Mini-DB evaluation. CALM uses the LLM as a lightweight surrogate to estimate QPS and recall feasibility for the generated candidates. It then selects a small batch of promising and diverse candidates for actual MiniDB execution. Only the selected candidates are executed; measured results establish feasibility and update the evaluation history and feasible archive. The loop continues until the tuning budget is exhausted,

and CALM returns its feasible non-dominated archive to the cross-MiniDB aggregation stage. If no feasible configuration has been observed, the local archive is empty.

5.2 Theoretical Analysis

We study CALM’s consistency under sustained exploration, following [26]. Let $\mathcal{S}_T$ contain the configurations actually evaluated during initialization and the first T search rounds. Write Q_T^* and Q^* for the best observed and optimal feasible QPS, respectively, with $Q_T^* = -\infty$ until a feasible configuration is found.

THEOREM 2 (CONSISTENCY OF CALM). *Consider a finite admissible space $\mathcal{X}$ with at least one feasible configuration and exact, stationary MiniDB evaluations. Suppose that, until first evaluated, each configuration c has conditional probability at least $p_c > 0$ of receiving a real evaluation in each round, uniformly over rounds and preceding search histories. For $\delta \geq 0$, let $\mathcal{G}_\delta$ contain the feasible configurations whose QPS is at least $Q^* - \delta$, and set $p_\delta = \max_{c \in \mathcal{G}_\delta} p_c$. Then, for any fixed T ,*

$$\Pr(\mathcal{S}_T \cap \mathcal{G}_\delta = \emptyset) \leq (1 - p_\delta)^T \leq e^{-p_\delta T}. \quad (11)$$

In an indefinitely continued run, an optimal feasible configuration is evaluated after finitely many rounds almost surely; hence,

$$Q_T^* \longrightarrow Q^* \quad \text{almost surely.} \quad (12)$$

If the feasible archive retains all non-dominated QPS–recall vectors without capacity truncation, it also equals the true feasible Pareto front after finitely many rounds almost surely.

PROOF. Repeated conditioning gives $\Pr(c \notin \mathcal{S}_T) \leq (1 - p_c)^T$ for every configuration c . Applying this bound to a member of $\mathcal{G}_\delta$ attaining p_δ proves the discovery bound, without requiring independent rounds. For $\delta = 0$, the failure probability tends to zero, so an optimal feasible configuration is eventually evaluated and its QPS remains the best observed value. The same argument applies to each of the finitely many Pareto-optimal configurations. Once all are evaluated, every other feasible objective vector is dominated by a true Pareto vector, so the complete non-dominated archive equals the true front. $\square$

The reachability condition covers region selection, candidate generation, and surrogate screening through their combined probability of real evaluation. The result characterizes asymptotic consistency on a MiniDB; finite-budget efficiency is assessed experimentally, and full-database performance is checked during final validation.

6 EVALUATION

This section presents the experimental results comparing μ -Tune against the state-of-the-art tuning approaches. The main results (Section 6.2) show that our method achieves the best performance across all datasets while requiring significantly less tuning time (addressing L1). The generalizability study (Section 6.3) and results under drift settings (Section 6.4) further demonstrate its transferability across diverse tuning scenarios (addressing L2). Finally, the ablation study (Section 6.5), sensitivity and robustness analysis (Section 6.6), and overhead analysis (Section 6.7) collectively evaluate the contributions of individual design choices, robustness across different settings, and associated computational costs. The μ -Tune implementation is available as open-source¹.

6.1 Experimental Setup

Workloads. We generate workloads using vector-db-benchmark [25] and test six representative real-world datasets: Nytimes², Glove³, GIST⁴, Tiny1M⁵, Msong⁶, and Geo-radius⁷ [2]. The basic information of these datasets are provided in Table 2.

Table 2: Evaluated Datasets.

Dataset	Size of BD	Size of QD	Dimension	Distance	Type
Nytimes	290000	1000	960	Cosine	Text
GloVe	1183514	10000	100	Cosine	Text
GIST	1000000	1000	96	L2	Image
Tiny5M	1000000	1000	384	L2	Image
Msong	994185	200	420	L2	Audio
Geo-radius	100000	1000	2048	Cosine	Synthetic

Hardware and Software. We evaluate μ -Tune on Milvus v2.3.1 [31], while Qdrant v1.16.0 [24] and pgvector v0.6.2 [23] is used only in the adaptability study. All experiments are conducted on identically configured machines, each equipped with a 16-core 3.60 GHz CPU and 64 GB of RAM.

Baselines. μ -Tune is compared with the following methods:

- **GridSearch** [32] sets the vector database according to predefined parameter configurations to search for the best one.
- **RandomSearch** uses Latin Hypercube Sampling (LHS) [18] to randomly sample a small number of parameter configurations to obtain the optimal parameter configuration.
- **VDTuner** [33] employs the Gaussian process regression and Constrained Bayesian optimization to auto-tune vector database.
- **PGTuner** [8] utilize deep reinforcement learning to recommend proximity graph configurations.
- **CHAT** [5] is an HNSW-specific tuner that leverages the index structure for deterministic configuration search and early pruning of resource-infeasible configurations.

Tuning Settings. As shown in Table 3, evaluated parameters for

¹<https://anonymous.4open.science/r/mu-tune/>

²<https://ann-benchmarks.com/nytimes-256-angular.hdf5>

³<http://ann-benchmarks.com/glove-100-angular.hdf5>

⁴<http://ann-benchmarks.com/gist-960-euclidean.hdf5>

⁵<https://www.cse.cuhk.edu.hk/systems/hash/gqr/dataset/tiny5m.tar.gz>

⁶<https://www.cse.cuhk.edu.hk/systems/hash/gqr/dataset/msong.tar.gz>

⁷https://storage.googleapis.com/ann-filtered-benchmark/datasets/random_geo_1m.tgz

Table 3: Index types and corresponding parameters.

Supported Index	Description	Building & Searching Parameters
AUTOINDEX	Default configuration	N/A ; N/A
FLAT	Exhaustive approach	N/A ; N/A
HNSW [19]	Graph-based	M, efConstruction ; ef
IVF FLAT [12]	Quantization-based	nlist ; nprobe
IVF SQ8 [12]	Quantization-based	nlist ; nprobe
IVF PQ [13]	Quantization-based	nlist, m, nbits ; nprobe
SCANN [10]	Quantization-based	nlist ; nprobe, reorder k

performance tuning are 16-dimensional in total, including index type, 8 index parameters and 7 system parameters. For GridSearch, we discretize the range of each parameter to construct a predefined coarse grid comprising 4068 complete configurations. Following prior work [8], RandomSearch employs stratified LHS to sample 14 index-construction configurations for each index type, resulting in 98 configurations in total. For VDTuner, we set the tuning budget to 200 rounds, consistent with its original paper. As PGTuner and CHAT support only graph-based index tuning, parameters outside its supported search space retain their default values.

Evaluation Metric. We evaluate the tuning performance of μ -Tune and the baselines in terms of tuning effect and tuning efficiency. Tuning effectiveness is evaluated by the QPS achieved by the configuration recommended for each recall constraint, with recall targets ranging from 0.85 to 0.99 in increments of 0.025. Tuning efficiency is measured by the end-to-end time required to complete the tuning process, referred to as the tuning time.

6.2 Tuning Performance Evaluation.

The experimental results prove that our method outperforms existing state-of-the-art in both *Effect* and *Efficiency*.

6.2.1 Tuning Effect Comparison. We compare the tuning effect of μ -Tune with all baselines. Figure 5 reports the maximal QPS achieved by different methods on different datasets and target recalls. Overall, μ -Tune consistently achieves the best or comparable QPS across all dataset-recall combinations, demonstrating its effectiveness in identifying high-quality configurations. Compared with GridSearch, the most competitive baseline for tuning effect, μ -Tune achieves average QPS improvements of 1.98%, 2.53%, 3.12%, 3.48%, 2.89%, 7.13% on NYTimes, GloVe, GIST, Tiny5M, Msong, and Geo-radius, respectively. Vector database tuning involves a large number of parameters, many of which have continuous or large-valued domains. Consequently, a practically affordable grid can only cover a limited and discretized subset of the configuration space, making it difficult for GridSearch to consistently identify high-quality configurations. The advantage of μ -Tune is more pronounced over RandomSearch, with an average QPS improvement of 94.53%. CHAT can also be regarded as a search-based method, but it focuses on a smaller set of HNSW-specific parameters. This restricted search space enables CHAT to explore these important parameters more thoroughly and thus achieve relatively strong performance; however, it also limits the potential for further optimization beyond index-specific configurations. As a result, CHAT achieves 12.58% lower QPS on average than μ -Tune. PGTuner, an RL-based method, faces a similar limitation: it optimizes only index-related parameters, thereby restricting the scope of configurations that can be explored. μ -Tune consequently outperforms PGTuner by

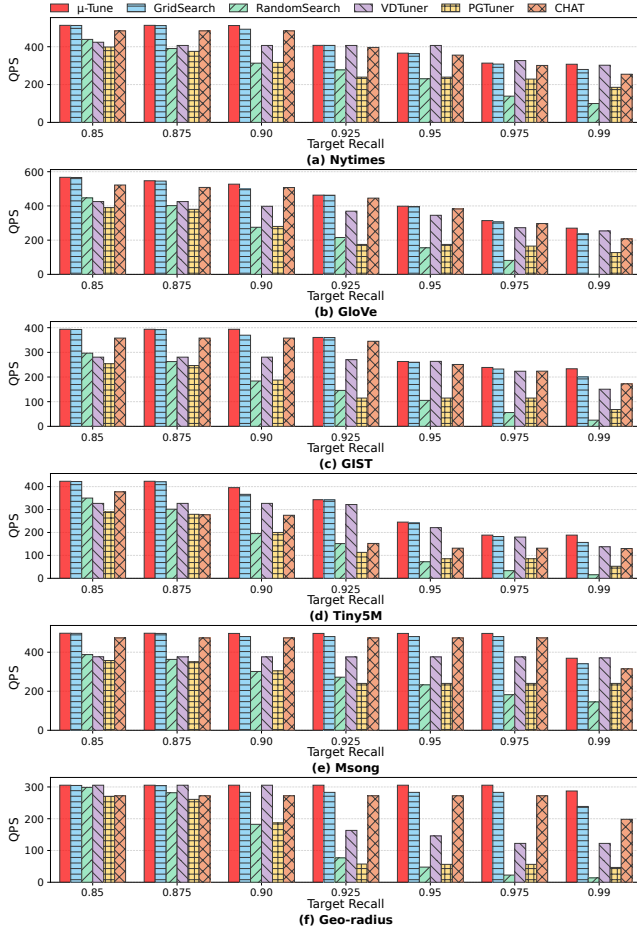


Figure 5: The QPS for all recommended configurations at different target recalls.

88.85% on average. Furthermore, μ -Tune achieves an average QPS improvement of 32.72% over VDTuner. The relatively suboptimal performance of VDTuner stems from the difficulty of accurately modeling the complex dependencies among configuration parameters, query performance, and tuning objectives, which may cause it to overlook promising configurations and produce less stable tuning results.

6.2.2 Tuning Efficiency Comparison. In this section, we compare the tuning time required by different methods. As shown in Figure 6 (a), μ -Tune substantially reduces the tuning time across all datasets, with reductions ranging from 37.10% to 99.15%. Compared with the second-fastest baseline, μ -Tune achieves an average speedup of 2.25 $\times$ across the evaluated datasets. The advantage is even more pronounced over GridSearch, which also achieves relatively strong tuning performance: GridSearch requires 67.00 $\times$, 118.00 $\times$, 110.43 $\times$, 93.70 $\times$, 73.38 $\times$, and 92.34 $\times$ more tuning time than μ -Tune on NY-Times, GloVe, GIST, Tiny5M, Msong, and Geo-radius, respectively, while μ -Tune achieves comparable or better query performance.

Figure 6 (b) further explains the source of this efficiency improvement. μ -Tune requires substantially fewer tuning iterations, while its average time per iteration remains lower than or comparable

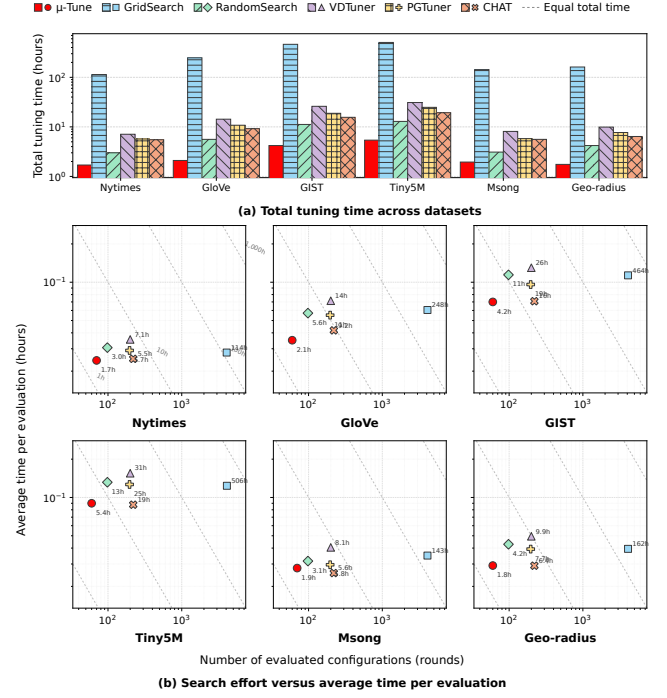


Figure 6: Tuning time comparison across different methods. (a) Total tuning time on different datasets. (b) Number of tuning iterations (x-axis) and average time per iteration (y-axis) for each method.

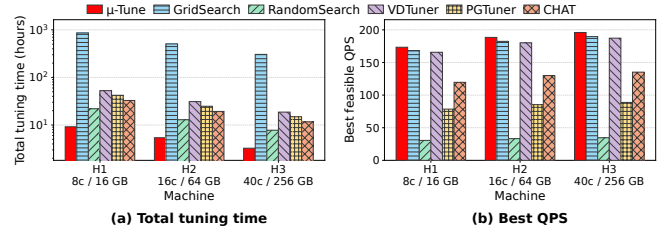


Figure 7: Hardware adaptability of different tuning methods. (a) Maximum QPS achieved under different hardware settings. (b) Tuning time required under different hardware settings.

to that of existing methods. The former benefit primarily comes from our LLM-based optimization algorithm, which enables more sample-efficient exploration of the configuration space, whereas the latter is mainly attributed to the MiniDB design, which reduces the cost of evaluating each candidate configuration. Specifically, μ -Tune requires only 60 tuning iterations on average to identify the recommended configuration, with an average cost of merely 0.046 hours per iteration. Overall, these results demonstrate that μ -Tune can identify high-quality configurations with substantially lower tuning overhead.

6.3 Generalizability Study

In this section, we delve into the generalizability of μ -Tune by expanding our evaluation framework to encompass new hardware environments and database engines.

Table 4: Tuning spaces of vector database engines.

Engine	Category	Tunable Parameters
Milvus	Index	AUTO/FLAT; HNSW (M, efConstruction); IVF-FLAT/SQ8 (nlist); IVF-PQ (nlist, m, nbits); SCANN (nlist)
	Search System	HNSW (ef); IVF-* (nprobe); SCANN (nprobe, reorder_k) gracefulTime, minSegmentSizeToEnableIndex, segment, maxSize, segment, sealProportion, insertBufSize, autoHandoff, autoBalance
Qdrant	Index	HNSW (m, ef_construct); quantization (quantile, compression, encoding, always_ram)
	Search System	hnsw_ef, exact, rescore, oversampling, ignore_full_scan_threshold, max_indexing_threads, on_disk_payload_m, inline_storage, on_disk_payload, default_segment_number, max_segment_size, mmap_threshold, indexing_threshold, flush_interval_sec, max_optimization_threads
pgvector	Index	HNSW (m, ef_construction); IVFFlat (lists)
	Search System	HNSW (ef_search); IVFFlat (probes) shared_buffers, effective_cache_size, maintenance_work_mem, max_wal_size, work_mem, effective_io_concurrency, random_page_cost, max_parallel_workers, max_parallel_maintenance_workers

6.3.1 Hardware. As detailed in Section 6.1, our primary evaluations are conducted on a server equipped with a 16-core CPU and 64 GB of RAM. To evaluate the adaptability of μ -Tune across different hardware environments, we additionally deploy the database on two machines with substantially different resource capacities: one equipped with 8 CPU cores and 16 GB of RAM, and the other with 40 CPU cores and 256 GB of RAM. We denote these three hardware environments, in ascending order of resource capacity, as H1, H2, and H3, respectively. Figure 7 reports the experimental results.

As shown in Figure 7 (a), the absolute QPS achieved by different methods varies considerably across hardware environments, confirming that hardware resources have a substantial impact on database performance and the resulting tuning outcomes. Nevertheless, μ -Tune consistently achieves the best or comparable QPS across H1–H3, outperforming the baselines by 3.3%–466.4% on average. This demonstrates that the effectiveness of μ -Tune is not tied to a particular hardware configuration and can generalize across both resource-constrained and resource-rich environments. Figure 7 (b) further compares the tuning time. Despite the large variation in available computational resources, μ -Tune consistently requires the shortest tuning time across all three hardware settings, achieving an average speedup of 2.39 $\times$ over the second-fastest baseline. In particular, its efficiency advantage remains evident on both H1 and H3, indicating that μ -Tune does not rely on abundant hardware resources to achieve efficient tuning. Overall, these results demonstrate that μ -Tune maintains both strong tuning effectiveness and efficiency across substantially different hardware environments, highlighting its adaptability to heterogeneous deployment settings.

6.3.2 DB Engine. Our primary experiments are conducted on Milvus v2.3.1. We further evaluate it on Qdrant v1.16.0 and pgvector v0.6.2 using the Tiny5M dataset to evaluate whether μ -Tune can generalize across different vector database engines. As summarized in Table 4, Milvus, Qdrant and pgvector expose substantially different tuning spaces. For μ -Tune, adapting to new database engines only requires specifying the corresponding tuning space and database execution interface; the underlying optimization framework remains unchanged and requires no pre-training or retraining. We

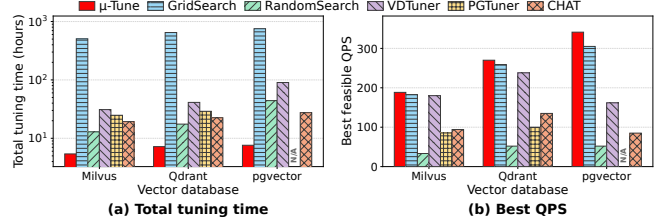


Figure 8: DB engine adaptability of different tuning methods. (a) Maximum QPS achieved under different vector database engines. (b) Tuning time required under different vector database engines.

set the target recall to 0.95 and compare the tuning effectiveness and efficiency of different methods. The results are reported in Figure 8.

Figure 8 (a) reports the best QPS achieved by different methods on the three database engines. μ -Tune consistently achieves the best or comparable query performance on both Milvus and Qdrant. On Qdrant, μ -Tune achieves 4.2% higher QPS than the second-best baseline, demonstrating that its tuning effectiveness is not tied to the parameter space or implementation of a specific vector database engine. Figure 8 (b) further compares the total tuning time. μ -Tune remains the most efficient method after switching from Milvus to Qdrant, requiring only 7.2 hours of tuning time and achieving a 2.43 $\times$ speedup over the second-fastest baseline. This result indicates that the efficiency benefits of our framework are also preserved across database engines. Overall, these results demonstrate that μ -Tune can readily adapt to vector database engines with substantially different configuration spaces while maintaining both high tuning effectiveness and efficiency.

pgvector represents a more challenging transfer setting. Unlike these purpose-built vector database systems, pgvector is implemented as an extension to the general-purpose PostgreSQL DBMS and therefore exposes a substantially different tuning interface. Consequently, some assumptions made by existing vector database tuners no longer hold. For example, PGTuner relies on an internal distance-count metric that is not supported by pgvector and thus cannot be applied, as indicated by “N/A” in Figure 8. Moreover, pgvector is considerably more sensitive to system configurations: failing to jointly configure PostgreSQL system knobs and vector-index parameters can severely limit its query performance, as illustrated by the poor result of CHAT. The larger and more intricate configuration space also makes it difficult for VDTuner to efficiently identify high-quality configurations. In contrast, μ -Tune leverages the LLM’s prior knowledge of PostgreSQL and pgvector configuration semantics to quickly identify promising regions and ultimately achieves the highest feasible QPS by a clear margin. The efficiency advantage of μ -Tune becomes even more evident on pgvector. Because pgvector executes large-scale vector workloads more slowly than the two purpose-built engines, evaluating each configuration on the full database becomes substantially more expensive, leading to markedly increased tuning costs for the baselines in Figure 8(b). μ -Tune largely avoids this overhead by performing most configuration evaluations on MiniDBs. Consequently, its total tuning time increases only marginally from Qdrant to pgvector and remains the lowest among all applicable methods. Overall, these

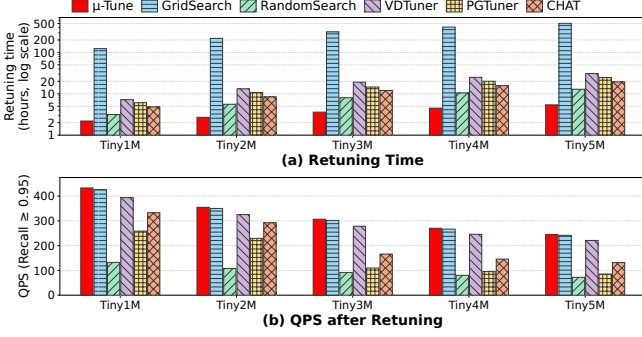


Figure 9: Tuning performance under data drift. (a) Maximum QPS achieved by different tuning methods. (b) Tuning time of different methods across increasing database sizes.

results demonstrate that μ -Tune generalizes not only across different vector database engines, but also to a fundamentally different DBMS architecture, while preserving both tuning effectiveness and efficiency.

6.4 Tuning Performance Comparison in Drift Scenarios.

In real-world deployments, vector databases are rarely static and may experience distributional changes over time. Such drift can lead to performance degradation of previously optimal configurations. In this section, we simulate two common types of drift, *data drift* and *query drift*, to evaluate the robustness and adaptability of μ -Tune under changing workloads. Following prior work, we set the target recall to 0.95.

6.4.1 Data Drift. In this section, we extract subsets containing 1M, 2M, 3M, and 4M vectors from Tiny5M, denoted as Tiny1M, Tiny2M, Tiny3M, and Tiny4M, respectively. Together with the original Tiny5M dataset, these datasets represent a gradually increasing database size. We therefore perform tuning sequentially as the database grows from Tiny1M to Tiny5M.

Figure 9 (a) reports the maximum QPS achieved by each method on the continuously growing Tiny datasets. μ -Tune achieves optimal or near-optimal QPS across all dataset sizes. These results demonstrate that μ -Tune can maintain strong tuning effectiveness as the underlying database continuously evolves. Figure 9 (c) further compares the tuning time of different methods. μ -Tune consistently achieves the shortest tuning time across all dataset sizes, mainly because it requires neither pre-training nor a large number of tuning iterations. More importantly, the efficiency advantage of μ -Tune becomes increasingly pronounced as the database grows. On Tiny1M, μ -Tune is 1.43 $\times$ faster than the second-fastest method. When the database size increases by five times from Tiny1M to Tiny5M, the tuning time of μ -Tune increases by only 2.45 $\times$, whereas that of the baselines increases by 4.12 $\times$ on average. Consequently, μ -Tune achieves a 2.39 $\times$ speedup on Tiny5M. This favorable scalability primarily stems from the MiniDB design. When the original database is relatively small, the reduction in evaluation cost brought by MiniDBs is less pronounced. As the database size grows, however, evaluating configurations directly on the full database becomes increasingly expensive, while MiniDBs keep the per-configuration

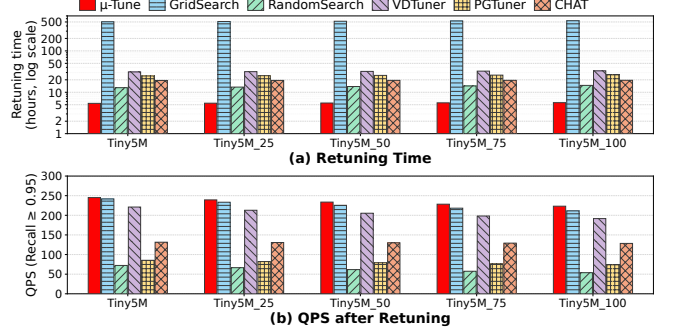


Figure 10: Tuning performance under query drift. (a) Maximum QPS achieved by different tuning methods. (b) Tuning time of different methods across increasing query-drift levels.

Table 5: Preliminary fidelity of Mini DBs in preserving the configuration–performance landscape. Values are mean $\pm$ sample standard deviation over three independently constructed Mini DBs.

Dataset	Sampling	Rank Corr. $\uparrow$	Recall MAE $\downarrow$	False-safe $\downarrow$	Regret@5 $\downarrow$
Tiny5M	Uniform	0.745 ± 0.024	0.031 ± 0.003	0.208 ± 0.038	0.117 ± 0.033
	μ -Tune	0.823 ± 0.016	0.024 ± 0.002	0.129 ± 0.034	0.071 ± 0.021
GloVe	Uniform	0.666 ± 0.039	0.044 ± 0.005	0.282 ± 0.054	0.181 ± 0.054
	μ -Tune	0.788 ± 0.026	0.031 ± 0.003	0.169 ± 0.041	0.096 ± 0.029
Msong	Uniform	0.705 ± 0.035	0.038 ± 0.004	0.250 ± 0.044	0.151 ± 0.041
	μ -Tune	0.799 ± 0.024	0.028 ± 0.003	0.157 ± 0.032	0.086 ± 0.024

evaluation cost substantially lower. As a result, the efficiency advantage of μ -Tune becomes more significant at larger scales, making it particularly well suited to modern large-scale vector database applications.

6.4.2 Query Drift. To simulate continuous query drift, we perturb the queries in Tiny5M by adding Gaussian noise at levels of 25%, 50%, 75%, and 100%, resulting in four drifted workloads denoted as Tiny5M_25, Tiny5M_50, Tiny5M_75, and Tiny5M_100, respectively. Accordingly, we sequentially perform tuning from Tiny5M_25 to Tiny5M_100.

Figure 10 (a) compares the maximum QPS achieved by different methods under query drift. μ -Tune consistently achieves the best tuning performance across all drift levels, demonstrating its ability to maintain high-quality tuning results as the query workload continuously evolves. Figure 10 (b) further compares the tuning time of different methods. μ -Tune remains the most tuning-efficient method across all query-drift scenarios, showing that its efficiency advantage is preserved even under continuously changing query distributions.

6.5 Ablation Study

We conduct ablation experiments to examine the two key components of μ -Tune: the Mini DB framework and CMOOLLN. Unless otherwise stated, all experiments use Milvus and Tiny5M on the testbed described in Section 6.1, with the recall constraint set to 0.95.

6.5.1 Fidelity of Mini Databases. We first examine whether Mini DBs preserve the configuration–performance landscape of the original database. For each dataset, we construct an evaluation set of 70 configurations using stratified Latin hypercube sampling, with ten configurations drawn for each supported index type. We compare our structure-aware sampling method with uniform random sampling. For each method, we independently construct three Mini DBs of the same size and evaluate the same configurations on both the Mini DBs and the original database.

Table 5 reports four metrics. First, we compute a constraint-aware version of Spearman rank correlation [27] between the configurations evaluated on a Mini DB and those evaluated on the original database, where feasible configurations are ranked by QPS and infeasible configurations are placed after all feasible ones. The correlation is first computed within each index type and then macro-averaged. Second, we report the mean absolute error in recall. Third, we measure the false-safe rate, i.e., the fraction of configurations that appear feasible on a Mini DB but violate the recall constraint on the original database. Finally, we report transfer regret at five, defined as the relative QPS gap between the best feasible configuration among the five candidates selected by a Mini DB and the best-known feasible configuration in the evaluation set.

Our sampling method consistently provides a more faithful approximation of the original tuning landscape. Across the evaluated datasets, it achieves an average rank correlation of 0.803, compared with 0.705 for uniform sampling. The corresponding recall error decreases from 0.038 to 0.028. More importantly, structure-aware sampling reduces the false-safe rate from 24.7% to 15.2%. This result is particularly important for constrained tuning because a false-safe configuration could be selected as the final recommendation despite violating the user-specified recall requirement on the original database. The improved fidelity also translates into better configuration transfer: our method obtains a Regret5 of 0.084, whereas uniform sampling incurs 0.150. These results demonstrate that preserving the tuning landscape, rather than only the marginal data distribution, is essential for reliable configuration transfer.

6.5.2 End-to-End Contribution of Mini Databases. We next evaluate the end-to-end contribution of Mini DBs by comparing four variants. DIRECT-FULL applies CMOOLLN directly to Tiny5M without constructing Mini DBs. UNIFORM-MINI replaces our sampling method with uniform sampling while retaining all other components. MEAN-ONLY uses the Mini DBs constructed by our method but ranks candidates only by their average normalized performance, removing the cross-database stability term. Finally, μ -TUNE denotes the complete framework. We consider two budget settings: (a) the equal-evaluation-budget setting, each method receives 60 tuner evaluations; and (b) the equal-wall-clock-budget setting, all methods are compared at the paired completion time of μ -TUNE. We repeat each experiment three times and report the mean and sample standard deviation.

As shown in Figure 11, under the equal-evaluation-budget setting, μ -TUNE achieves 245.14 QPS, compared with 248.17 QPS for DIRECT-FULL. The difference is only 1.22%, showing that tuning on substantially smaller Mini DBs preserves most of the configuration quality obtainable through direct full-scale tuning. The advantage

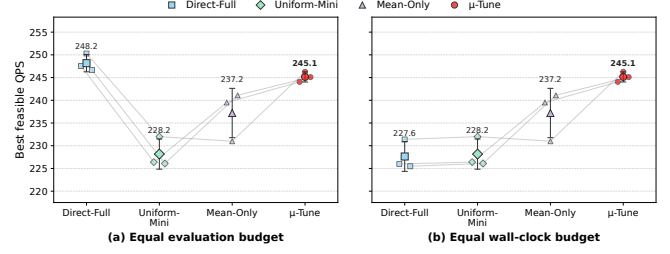


Figure 11: End-to-end ablation of Mini DBs. (a) an equal evaluation budget and (b) an equal wall-clock budget, with recall constrained to at least 0.95.

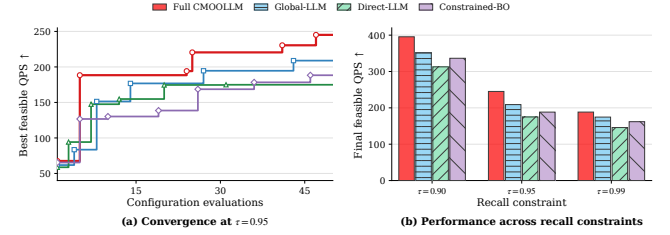


Figure 12: Ablation of the design choices in CMOOLLN. (a) Convergence curves under a recall constraint of 0.95. (b) Feasible QPS across different recall constraints.

becomes more pronounced under the equal-wall-clock-budget setting. Within the time required by μ -TUNE, DIRECT-FULL reaches only 227.63 QPS, while μ -TUNE achieves 245.14 QPS, corresponding to an improvement of 7.69%. Moreover, replacing structure-aware sampling with uniform sampling reduces the QPS from 245.14 to 228.17, a decrease of 7.44%. Removing the stability term reduces the final QPS from 245.14 to 237.20 and increases the across-run sample standard deviation from 1.11 to 5.43 QPS. Thus, both structure-aware sampling and cross-database stability contribute to reliable end-to-end tuning.

6.5.3 Design Choices of CMOOLLN. We investigate whether the effectiveness of CMOOLLN stems merely from the use of an LLM or from its structured optimization workflow. We compare the complete CMOOLLN with three variants. GLOBAL-LLM removes the region partitioning mechanism and generates configurations over the entire heterogeneous search space, while retaining the same iterative feedback loop as CMOOLLN. DIRECT-LLM further removes the structured optimization workflow and directly prompts the same LLM at each iteration to recommend the next candidate configurations based only on the raw evaluation history. CONSTRAINED-BO replaces the LLM-based optimizer with a conventional constrained Bayesian optimizer using the same search space. We conduct the experiment on Tiny5M under recall constraints of $\{0.90, 0.95, 0.99\}$.

Figure 12(a) presents the convergence curves under $\tau = 0.95$. Full CMOOLLN maintains a higher best-so-far feasible QPS throughout most of the tuning process and ultimately achieves the highest QPS among all methods. In comparison, GLOBAL-LLM improves progressively through iterative feedback but remains below CMOOLLN throughout most of the tuning process. This gap demonstrates the benefit of partitioning the heterogeneous, index-specific configuration space into more coherent regions, which allows the optimizer to concentrate its search on promising subspaces. DIRECT-LLM

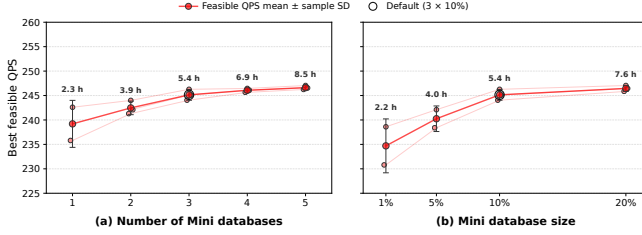


Figure 13: Sensitivity Analysis of Mini Databases.

Table 6: Performance and API cost for μ -Tune.

Model	Reasoning Effort	Best QPS	LLM Time (s)	Token Usage (Input/Output/Total)	Token Cost (USD)
DeepSeek-V4-Pro-813	high	322.11	269.75	31,938/76,546/108,484	0.2590
gpt-5.4-2026-03-05	high	342.68	296.48	43,443/46,692/90,135	0.8090
gpt-5.4-2026-03-05	medium	315.84	98.64	52,167/17,212/69,379	0.3886
gpt-5.4-2026-03-05	low	305.29	70.50	52,792/13,770/66,562	0.2854
claude-opus-4-7	adaptive	356.33	255.20	40,882/54,039/94,921	1.4588
gpt-5.6-sol	high	359.34	235.18	43,079/38,006/81,085	0.9320

achieves a relatively strong result within the first few evaluations but quickly reaches a performance plateau. This behavior indicates that the prior knowledge encoded in the LLM is useful for identifying reasonable configurations, but few-shot recommendations cannot exploit measured database feedback to refine them toward the high-performance region. CONSTRAINED-BO exhibits slower improvement and achieves only moderate final performance under the same evaluation budget. Although it can gradually learn the relationship between configurations, recall, and QPS, it requires more observations to model the mixed and conditional search space effectively. Figure 12(b) reports the feasible QPS under different recall constraints. CMOOLLN consistently achieves the best result across all three thresholds. These results show that CMOOLLN’s effectiveness does not arise solely from introducing an LLM into the tuning process.

6.6 Sensitivity and Robustness Analysis

We next study whether the effectiveness of μ -Tune is sensitive to its main design parameters or the choice of LLM backbone.

6.6.1 Number and Size of Mini Databases. We first vary the number of Mini DBs over 1, 2, 3, 4, 5 while fixing the size of each Mini DB to 10% of Tiny5M. We then vary the sampling ratio over 1%, 5%, 10%, 20%, while fixing the number of Mini DBs to three.

Figure 13 shows the results. Increasing the number of Mini DBs from one to three improves the final QPS from 239.20 to 245.14 and reduces the standard deviation across feasible runs from 4.81 to 1.11. This improvement shows that multiple independent Mini DBs reduce the risk of selecting configurations that perform well only on a particular sample. Increasing the number further to five yields only an additional 0.60% improvement, while increasing tuning time by 56.85%. Thus, three Mini DBs provide a favorable balance between transfer reliability and tuning cost. A similar saturation trend is observed for Mini DB size. Increasing the sampling ratio from 1% to 10% improves QPS by 4.45%. Increasing the ratio to 20%, however, provides only a further 0.53% improvement at an additional tuning-time cost of 40.56%. These results support the default setting of three 10% Shadow DBs, which lies near the knee of the quality–cost curve.

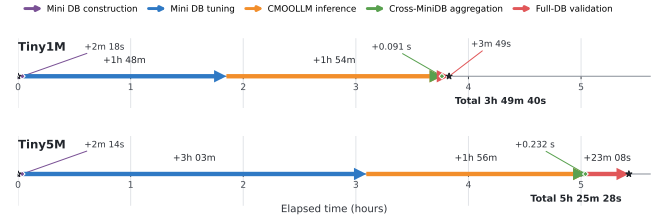


Figure 14: End-to-end cost breakdown.

6.6.2 Robustness to LLM Backbones. Since μ -Tune employs an LLM to propose new configurations, we investigate whether its effectiveness depends on a particular LLM backbone or reasoning effort. We evaluate four representative models: GPT-5.4 [21], DeepSeek-V4-pro [7], Claude-opus-4.7 [1], and GPT-5.6-sol [22]. For GPT-5.4, we further vary the reasoning effort among *low*, *medium*, and *high*. We use the same GloVe dataset and a recall constraint of 0.95 for all backbones, while keeping All model-independent settings identical.

As shown in Table 6, μ -Tune consistently identifies high-quality feasible configurations with all backbones. This consistency indicates that the effectiveness of μ -Tune is not tied to a particular model or provider. We also observe a clear trade-off among configuration quality, inference latency, and monetary cost. More capable backbones and higher reasoning effort generally yield better candidate configurations and higher final feasible QPS, but also require longer inference time and incur greater API cost. Considering both tuning quality and operational cost, we therefore use GPT-5.4 with *medium* reasoning effort as the default setting in the main experiments.

6.7 Overhead Analysis

We finally analyze the end-to-end cost of μ -Tune to understand where its tuning cost is spent and whether its additional components introduce substantial overhead. We divide the total tuning time into five components: (1) Mini DB construction, including projection, sampling, data loading, and exact ground-truth computation; (2) Mini DB tuning, including index construction, configuration deployment, and query execution; (3) CMOOLLN inference, including prompt construction, API calls, response parsing, and retries; (4) cross-Shadow candidate aggregation; and (5) final full-database validation. Figure 14 reports the the resulting runtime breakdown on Tiny1M and Tiny5M.

MiniDB tuning and CMOOLLN inference account for the majority of the end-to-end runtime. Although the database size increases fivefold from Tiny1M to Tiny5M, the MiniDB tuning time on Tiny5M is only 1.69 $\times$ that on Tiny1M. This sublinear growth results from evaluating most configurations on substantially smaller MiniDBs rather than on the full database. Meanwhile, CMOOLLN inference time remains nearly unchanged because it is dominated by LLM calls and is largely independent of database size. Consequently, the total tuning time on Tiny5M is only 1.42 $\times$ that on Tiny1M. This favorable scaling suggests that the efficiency advantage of μ -Tune becomes more pronounced as database size increases. Moreover, MiniDB construction and cross-MiniDB candidate aggregation introduce negligible overhead on both datasets, confirming that their costs do not offset the savings achieved by MiniDB-based tuning.

7 CONCLUSION

We presented μ -Tune, an efficient and generalizable framework for end-to-end vector database tuning. It combines structure-preserving MiniDBs with region- and constraint-aware LLM optimization to reduce both evaluation costs and tuning iterations, while decoupling the core tuner from engine-specific details and supporting the system, index, and search layers. Experiments across diverse datasets, recall requirements, database engines, and hardware environments demonstrate that μ -Tune consistently finds high-quality feasible configurations with substantially lower tuning time than existing methods.

ACKNOWLEDGMENTS

The authors used ChatGPT to assist with English editing and code development. No experimental results or original contributions were generated solely by AI tools.

REFERENCES

- [1] Anthropic. 2026. Introducing Claude Opus 4.7. Retrieved September 7, 2026 from <https://www.anthropic.com/news/claude-opus-4-7>
- [2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87 (2020), 101374. <https://doi.org/10.1016/j.is.2019.02.006>
- [3] Baoqing Cai, Yu Liu, Ce Zhang, Guangyu Zhang, Ke Zhou, Li Liu, Chunhua Li, Bin Cheng, Jie Yang, and Jiashu Xing. 2022. HUNTER: An Online Cloud Database Hybrid Tuning System for Personalized Requirements. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) (SIGMOD '22). Association for Computing Machinery, New York, NY, USA, 646–659. <https://doi.org/10.1145/3514221.3517882>
- [4] Moses S. Charikar. 2002. Similarity estimation techniques from rounding algorithms. In *Proceedings of the Thirty-Fourth Annual ACM Symposium on Theory of Computing* (Montreal, Quebec, Canada) (STOC '02). Association for Computing Machinery, New York, NY, USA, 380–388. <https://doi.org/10.1145/509907.509965>
- [5] Geon Choi, Hoeun Lee, and Jaeyoung Do. 2026. Exploiting Structural Properties for Efficient Constraint-Aware HNSW Hyperparameter Tuning. *arXiv:2607.04630* [cs.DB]. <https://arxiv.org/abs/2607.04630>
- [6] William G. Cochran. 1977. *Sampling Techniques* (3rd ed.). John Wiley & Sons, New York.
- [7] DeepSeek. 2026. DeepSeek API: Models & Pricing. Retrieved September 7, 2026 from https://api-docs.deepseek.com/quick_start/pricing
- [8] Hao Duan, Yitong Song, Bin Yao, and Anqi Liang. 2025. PGtuner: An Efficient Framework for Automatic and Transferable Configuration Tuning of Proximity Graphs. *Proc. ACM Manag. Data* 3, 4, Article 261 (Sept. 2025), 27 pages. <https://doi.org/10.1145/3749179>
- [9] Michael A. Gelbart, Jasper Snoek, and Ryan P. Adams. 2014. Bayesian optimization with unknown constraints. In *Proceedings of the Thirtieth Conference on Uncertainty in Artificial Intelligence* (Quebec City, Quebec, Canada) (UAI'14). AUAI Press, Arlington, Virginia, USA, 250–259.
- [10] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning* (ICML'20). JMLR.org, Article 364, 10 pages.
- [11] Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30. <https://doi.org/10.1080/01621459.1963.10500830>
- [12] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>
- [13] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [14] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: Re-rank with source coding. *Acoustics, Speech, and Signal Processing*, 1988. ICASSP-88., 1988 International Conference on (02 2011). <https://doi.org/10.1109/ICASSP.2011.5946540>
- [15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '20). Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [16] Guoliang Li, Xuanhe Zhou, Shifu Li, and Bo Gao. 2019. Q-Tune: a query-aware database tuning system with deep reinforcement learning. *Proc. VLDB Endow.* 12, 12 (Aug. 2019), 2118–2130. <https://doi.org/10.14778/3352063.3352129>
- [17] Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. 2024. Large Language Models to Enhance Bayesian Optimization. *arXiv:2402.03921* [cs.LG]. <https://arxiv.org/abs/2402.03921>
- [18] Wei-Liem Loh. 1996. On Latin hypercube sampling. *Annals of Statistics* 24 (1996), 2058–2080. <https://api.semanticscholar.org/CorpusID:121050124>
- [19] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [20] Gabriele Ortonato. 2024. Bayesian Optimization for Hyperparameters Tuning in Neural Networks. *arXiv:2410.21886* [cs.LG]. <https://arxiv.org/abs/2410.21886>
- [21] OpenAI. 2026. GPT-5.4 Model. API documentation. Retrieved September 7, 2026 from <https://developers.openai.com/api/docs/models/gpt-5.4>
- [22] OpenAI. 2026. Previewing GPT-5.6 Sol: A Next-Generation Model. Retrieved September 7, 2026 from <https://openai.com/index/previewing-gpt-5-6-sol/>
- [23] pgvector contributors. [n.d.]. pgvector: Open-Source Vector Similarity Search for Postgres. Software and documentation, version 0.6.2. Retrieved September 7, 2026 from <https://github.com/pgvector/pgvector/tree/v0.6.2>
- [24] Qdrant. [n.d.]. Qdrant: Vector Similarity Search Engine. Software repository, version 1.16.0. Retrieved September 7, 2026 from <https://github.com/qdrant/qdrant/tree/v1.16.0>
- [25] Qdrant. [n.d.]. Vector-db-benchmark: Framework for benchmarking vector search engines. Software repository. Retrieved September 7, 2026 from <https://github.com/qdrant/vector-db-benchmark>
- [26] Andrej Schwanke, Lyubomir Ivanov, David Salinas, Frank Hutter, and Arber Zela. 2026. Multi-Objective Hierarchical Optimization with Large Language Models. *arXiv:2601.13892* [cs.LG]. <https://arxiv.org/abs/2601.13892>
- [27] C. Spearman. 2010. The proof and measurement of association between two things. *International Journal of Epidemiology* 39, 5 (10 2010), 1137–1150. <https://doi.org/10.1093/ije/dyq191> *arXiv:https://academic.oup.com/ije/article-pdf/39/5/1137/18481215/dyq191.pdf*
- [28] Ján Suchal and Pavol Návrát. 2010. Full Text Search Engine as Scalable k-Nearest Neighbor Recommendation System. In *Artificial Intelligence in Theory and Practice III*, Max Bramer (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 165–173.
- [29] Immanuel Trummer. 2022. DB-BERT: A Database Tuning Tool that “Reads the Manual”. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) (SIGMOD '22). Association for Computing Machinery, New York, NY, USA, 190–203. <https://doi.org/10.1145/3514221.3517843>
- [30] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) (SIGMOD '17). Association for Computing Machinery, New York, NY, USA, 1009–1024. <https://doi.org/10.1145/3035918.3064029>
- [31] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) (SIGMOD '21). Association for Computing Machinery, New York, NY, USA, 2614–2627. <https://doi.org/10.1145/3448016.3457550>
- [32] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. <https://doi.org/10.14778/3476249.3476255>
- [33] Tiannuo Yang, Wen Hu, Wangqi Peng, Yusen Li, Jianguo Li, Gang Wang, and Xiaoguang Liu. 2024. VDTuner: Automated Performance Tuning for Vector Data Management Systems. In *2024 IEEE 40th International Conference on Data Engineering* (ICDE). 4357–4369. <https://doi.org/10.1109/ICDE60146.2024.00332>
- [34] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) (SIGMOD '19). Association for Computing Machinery, New York, NY, USA, 415–432. <https://doi.org/10.1145/3299869.3300085>
- [35] Xinyi Zhang, Hong Wu, Zhuo Chang, Shuwei Jin, Jian Tan, Feifei Li, Tieying Zhang, and Bin Cui. 2021. ResTune: Resource Oriented Tuning Boosted by Meta-Learning for Cloud Databases. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) (SIGMOD '21). Association for Computing Machinery, New York, NY, USA, 2102–2114. <https://doi.org/10.1145/3299869.3300085>

- [//doi.org/10.1145/3448016.3457291](https://doi.org/10.1145/3448016.3457291)
- [36] Yuqing Zhu, Jianxun Liu, Mengying Guo, Yungang Bao, Wenlong Ma, Zhuoyue Liu, Kunpeng Song, and Yingchun Yang. 2017. BestConfig: tapping the performance potential of systems via automatic configuration tuning. In Proceedings of the 2017 Symposium on Cloud Computing (Santa Clara, California) (SoCC '17). Association for Computing Machinery, New York, NY, USA, 338–350. <https://doi.org/10.1145/3127479.3128605>